

NAME: ADEBIMPE ADEROJU M.

COURSE: CSE210- PROGRAMMING WITH FUNCTIONS

INSTRUCTURE'S NAME: JOHN READING

What is inheritance and why is it important?

Inheritance is a core principle of object-oriented programming that allows one class (called a child or derived class) to automatically receive all the attributes and methods from another class (called a parent or base class). This creates an "is-a" relationship where a more specific type inherits the general characteristics of a more general type. For example, a Student "is a" Person, so Student inherits Person's name and basic behaviors, then adds student-specific features like a student ID.

Key Benefit: The biggest advantage is code reuse. Instead of rewriting common functionality in every class, you define it once in the base class and all child classes get it for free. This saves development time, reduces duplication, and makes maintenance easier - when you fix something in the base class, all derived classes benefit automatically.

Real-world Application: In my Homework program, I modeled school assignments. Both MathAssignment and WritingAssignment share common traits (student name, topic) and behavior (GetSummary() method), but each has unique features. Rather than duplicating the shared code, I created an Assignment base class:

```
csharp // Base class with shared functionality public class Assignment { private string
_studentName; private string _topic;

public Assignment(string studentName, string topic)
{
    _studentName = studentName;
    _topic = topic;
}

public string GetSummary()
{
    return $"{_studentName} - {_topic}";
}

}

// Derived classes inherit + add specialization public class MathAssignment : Assignment // ←
Inheritance! { private string _textbookSection; private string _problems;

public MathAssignment(string studentName, string topic, string
textbookSection, string problems)
    : base(studentName, topic) // ← Calls parent constructor
```

```

{
    _textbookSection = textbookSection;
    _problems = problems;
}

public string GetHomeworkList()
{
    return $"Section {_textbookSection} Problems {_problems}";
}

```

} How it works: Notice `MathAssignment : Assignment` - the colon creates the inheritance relationship. The `base(studentName, topic)` line calls the parent constructor to set the inherited fields. Most importantly, `MathAssignment` automatically gets `GetSummary()` for free without writing it! When I call `math.GetSummary()`, it uses the base class method even though `MathAssignment` never defined it.

This demonstrates inheritance perfectly: common assignment logic lives in one place (`Assignment`), while specialized math homework details live in `MathAssignment`. The result is cleaner, more maintainable code that follows the DRY principle.